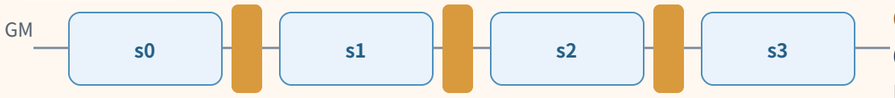
Roofline 给我们的启发：先减少高层存储往返，再平衡片上服务

固定 FFT 语义与计算核心 P：IL=WP/QL，吞吐 ≤ min(Ppeak, BWL · IL)。Roofline 指出流量方向，不替代依赖、容量与流水分析。

1 同一 N=16 FFT，计算不变，GM 物化边界不同

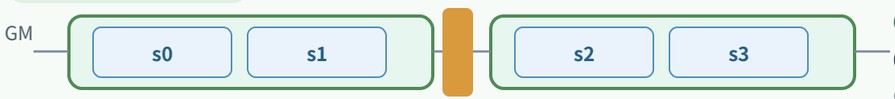
B=1，K=log2(N)=4，q=每个复数 payload 字节数；每个执行组完整读取并写出 16 个复数。

每级都物化到 GM



G=4
QGM=128q = 1024 B (q=8)
四次完整读写；中间边界反复付费

每两级在 UB 内驻留



G=2
QGM=64q = 512 B (q=8)
GM 算术强度相对上行提高 2×

四级全部合法驻留

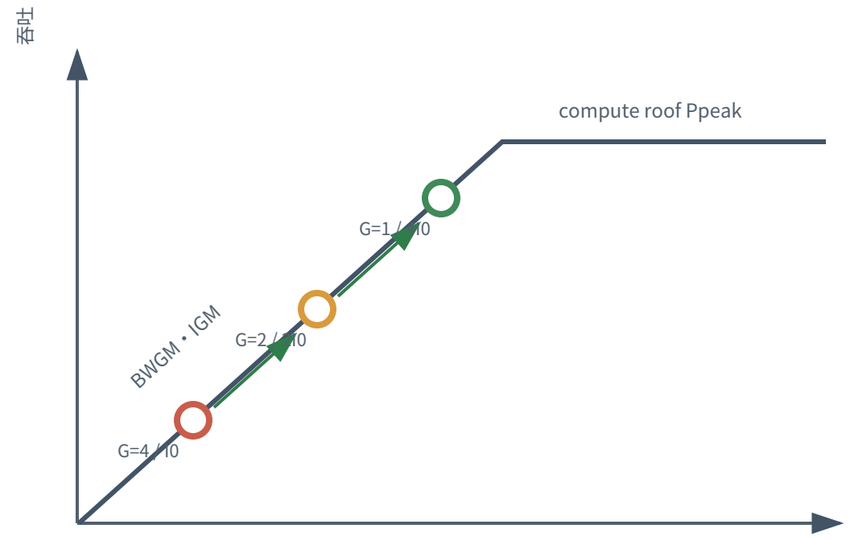


G=1
QGM=32q = 256 B (q=8)
只剩 compulsory input/output；强度提高 4×

橙色竖条 = 完整数组在 GM 物化；绿色外框 = 中间状态在所选片上层级保持。

2 流量减少怎样提高设计点的 Roofline 上界

横轴是 log2 GM 算术强度；空心点是上界投影，不是实测吞吐。



上界右移不代表实测点沿屋顶线移动
GM 瓶颈减轻后，UB 交换、Vector 发射、依赖等待或容量可能成为新上界。

符号化上界示意；点和折点没有使用 Ascend 实测坐标。

3 因此我们不是“盲目增加并行度”，而是平衡四类服务

先用 Roofline 找到应减少的高层流量，再用依赖、容量和 profile 决定哪种空间-时间分解能够兑现它。

A 计算物化边界 G
先问同一份有效计算为何反复经过 GM；
目标是减少完整数组写回/重读。

→

B 选择驻留层级 L
UB 能否容纳并持续供给 live state?
放不下或喂不满时，减少 G 不能兑现。

→

C 平衡空间复制与时间复用
固定 C=Us · Ud，重分配边界、内部链路、
状态容量和启动/尾部压力。

→

D 用计数器和同协议实测闭环
payload 只是逻辑账本；事务、重排、twiddle、
MTE/Vector 重叠与等待必须由 profile 验证。

Ud · 数据空间
增加独立数据并发；G 不变时
不会自动提高 IGM。
过大还会扩大边界与活跃状态。

Td / batch · 数据时间
摊销启动与填充、保持硬件繁忙；
简化 payload 模型下强度不变。
规模变大不消除每元素重复流量。

Us · 阶段空间
减少反馈/边界压力，但增加阶段间
交接、系数并发与 live state。
Us 不是“融合了几级”的别名。

Ts + residence · 阶段时间
复用同一阶段服务；只要状态仍留在
UB，就不必为每个 fold 支付 GM。
时间复用本身不等于低效。

layout / boundary
决定理论 payload 是否变成合并、
无冲突的实际事务。
在线重排有代价，需实测。

计算核心 P
改变 WP、Ppeak 与资源需求；
不自动消除物化边界 G。
与计算流优化基本正交、成本耦合。

方法推导来源：cuButterfly。N=16 为可核算教学例，不是 Ascend 实测 Roofline；当前实现状态与计数器证据以 Ascend-FFT 代码和归档为准。